

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«Принципы разработки операционных систем»

по дисциплине «Операционные системы»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Гавва Г.Д.

Санкт-Петербург
2026г.

1. Цель работы.

Изучение основ разработки ОС, принципов низкоуровневого взаимодействия с аппаратным обеспечением, программирования системной функциональности и процесса загрузки системы.

2. Задачи работы

1. Скомпилировать и запустить на эмуляторе пример загрузочного сектора start.asm с помощью fasm
2. Разработать простой загрузчик для загрузки минимального ядра. Скомпилировать загрузчик и минимальное ядро с помощью GCC и fasm. Проверить работоспособность загрузчика и ядра на эмуляторе.
3. Разработать функции загрузчика: загрузка ядра по адресу 0x10000, добавить поддержку считывания клавиш и отправки на ядро режима работы (VM или STD)
4. Разработать функции ядра: info, upcase, downcase, template, search
5. Придумать не меньше 15 тестов для проверки работоспособности ОС. Протестировать вашу реализацию на предложенных тестах
6. Ответить на контрольные вопросы

3. Описание решения

В начале был написан загрузчик по примерам из Теоретических сведений: стартовая инициализация, перевод ядра в защищенный режим и вызов минимального ядра на асемблере, взятого из методических указаний. Далее были проведены исправительные мероприятия и подгон кода под среду в которой проводится лабораторная работа, такие как изменение размера таблицы gdt на единицу, а также использование вместо инструкции call jmp на адрес 0x08:0x10000. Данные действия помогли избежать проблем с загрузкой и полностью передают управление ядру. Так же был разработан

блок загрузки ядра в память по нужному адресу с диска b. Блок-схема работы загрузчика изображена ниже(см. Рис. 1)

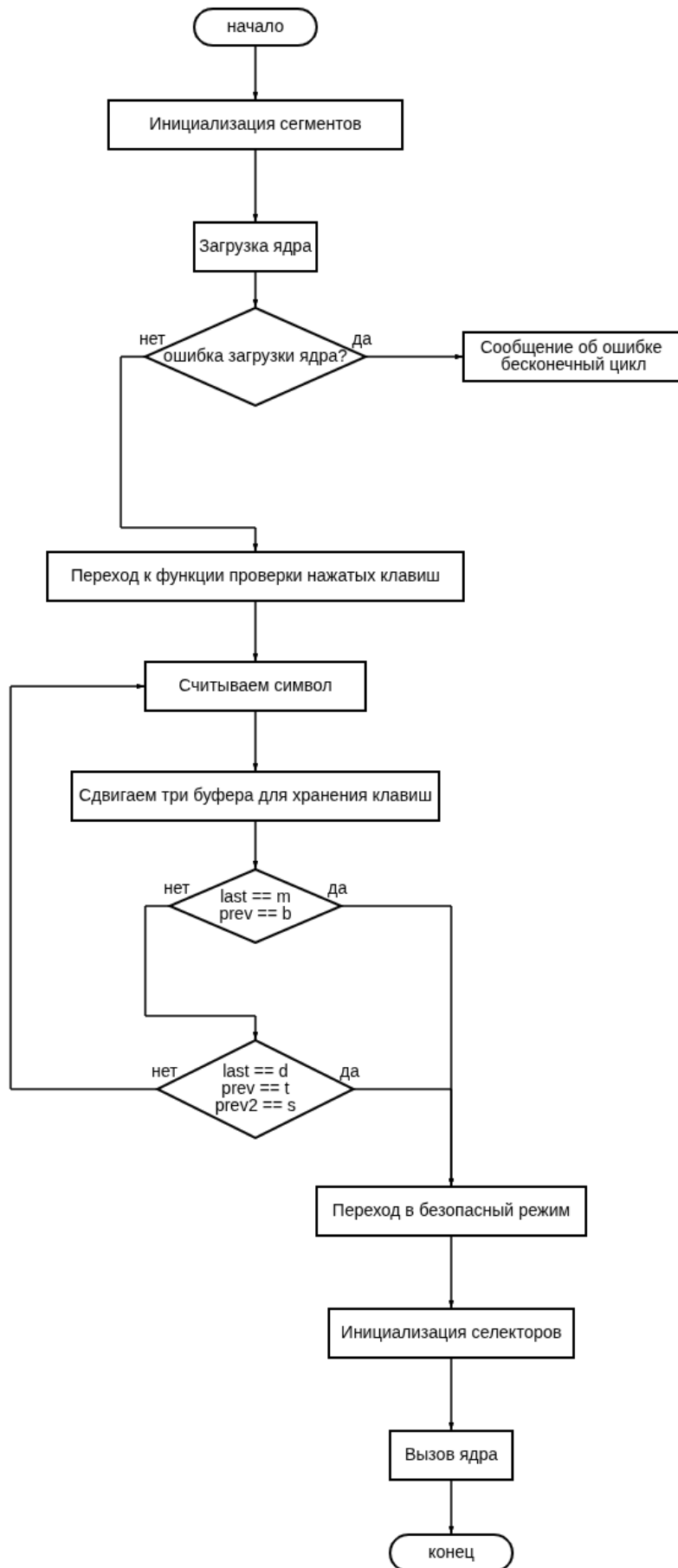


Рис. 1. Блок-схема работы загрузчика

За основу ядра для выполнения лабораторной работы был взят код из Теоретических сведений и объединенный в одну программу. После аналогично загрузчику были проведены исправительные мероприятия, чтобы данные куски кода работали исправно на современных системах и новой версии эмулятора.

После этих действий были созданы вспомогательные функции для выполнения лабораторной работы, которые не относились к основной реализации задания, но помогли в отладке и обеспечении масштабируемого кода в будущем.

В следующей таблице приведены описания работы основных функций ядра, которые использовались для выполнения задачи.

Название функции	Назначение
<code>void command_machine (unsigned char* str)</code>	Обрабатывает поступающие на вход команды и выполняет действия указанные в задании
<code>void to_upcase (unsigned char* in, unsigned char* out)</code>	Получает на вход функцию и путем вычитания числа 32 (потому что букв всего 27 и заглавные буквы идут до строчных) из каждого символа строки, получает строку в верхнем регистре
<code>void to_downcase (unsigned char* in, unsigned char* out)</code>	Аналогично функции to_upcase для каждого символа прибавляет 32 и получает строку в нижнем регистре
<code>void int_to_str (int value, unsigned char* buffer)</code>	Переводит число в его строковую форму, Например на вход число 100, на выходе строка «100»
<code>int std_search (unsigned char* text)</code>	Ищет первое вхождение подстроки в строке по алгоритму полного перебора каждой позиции в которой может быть совпадение
<code>int boyer_moore_search (unsigned char* text)</code>	Ищет первое вхождение подстроки в строке по алгоритму Бойера-Мура, используя эвристику плохих символов и хороших суффиксов, проводящая сравнение с конца строки, а не с начала, как в стандартном алгоритме
<code>void print_bm_table_simple (unsigned char* pattern)</code>	Выводит на экран информацию о сдвиге символов в соответствии с эвристикой плохих символов, где символу ставится сдвиг равный расстоянию от него до конца подстроки

Таблица 1. Функции использованные для выполнения лабораторной работы

4. Результаты работы

Были разработаны ядро и загрузчик в соответствии с заданием, создан интерфейс для масштабирования написания операционной системы. Протестированы основные функции ядра, представленные в методических указаниях. Результаты тестирования приведены в Таблица 2

Тест	Результат
1. В загрузчике было введено: !@#\$\$%^&*()bm	Переход в режим ВМ
2. В загрузчике было введено: stbm	Переход в режим ВМ
3. upcase	Ничего не вывелось, ошибки нету
4. downcase ! @#\$\$%^&*()1234567890	Вывело тоже самое, так как это не буквы
5. downcase HELLO	hello
6. upcase HeL Lo B y E	HEL LO B Y E
7. upcase HELLO	HELLO
8. upcase helooooooooooooooooooooo	HELLOooooooooooooooooooooo
9. template helooooooooooooooooooooo	Template «helooooooooooooooooooooo» loaded. BM info: h:30 e:29 l:1 o:31
10. template helooooooooooooooooooooo	Template «helooooooooooooooooooooo» loaded.
11. template	Template «» loaded. Empty pattern
12. template search	Found «» at pos: 0
13. template hello search world wrold world world worlhello	Found «hello» at pos: 28
14. template search	Found «» at pos: 0
15. template hello search world wrold world world worlhello	Found «hello» at pos: 28
16. info info info info info info	Экран очищается и выводится сверху информация о разработчике и режиме работы

Таблица 2. Входные данные для программ и результаты их работы

5. Выводы

Изучены принципы разработки операционных систем, низкоуровневого взаимодействия с аппаратным обеспечением. Запрограммирована системная функциональность и процесс загрузки системы.

Приложения

Приложение 1

run.sh

```
rm -f *.o bootsect.bin kernel_cpp.bin
```

```
fasm bootsect.asm bootsect.bin
```

```
# Компилируем каждый cpp файл
```

```
g++ -ffreestanding -fno-stack-protector -m32 -fno-pie -c main.cpp  
-o main.o
```

```
g++ -ffreestanding -fno-stack-protector -m32 -fno-pie -c  
screen.cpp -o screen.o
```

```
g++ -ffreestanding -fno-stack-protector -m32 -fno-pie -c  
keyboard.cpp -o keyboard.o
```

```
g++ -ffreestanding -fno-stack-protector -m32 -fno-pie -c  
commands.cpp -o commands.o
```

```
g++ -ffreestanding -fno-stack-protector -m32 -fno-pie -c  
strings.cpp -o strings.o
```

```
# Линкуем все объектные файлы вместе
```

```
ld -m elf_i386 -T linker.ld -o kernel_cpp.bin --oformat binary  
main.o screen.o keyboard.o commands.o strings.o
```

```
qemu-system-i386 -fda bootsect.bin -fdb kernel_cpp.bin
```

Приложение 2

bootsect.asm

```
use16
```

```
org 0x7C00
```

```
start:
```

```
    ; Инициализация сегментов
```

```
    xor ax, ax
```

```
    mov ds, ax
```

```
    mov es, ax
```

```
    mov ss, ax
```

```

mov sp, 0x7C00

; Вывод сообщения
mov si, msg
call print_string

; Загрузка ядра (сектор 2 и далее)
mov ah, 0x02      ; функция чтения
mov al, 15        ; читаем 4 сектора (2KB)
mov ch, 0         ; цилиндр 0
mov cl, 1         ; сектор 2 (начинаем со 2-го сектора)
mov dh, 0         ; головка 0
mov dl, 0x01

; Куда загружать
mov bx, 0x1000
mov es, bx
xor bx, bx

int 0x13
jc disk_error

; Переход к загруженному ядру
mov si, load_ok_msg
call print_string

call wait_for_keys

jmp turn_protected

wait_for_keys:
pusha

; Храним только последние 2 символа для bm
; и последние 3 для std
mov byte [last], 0
mov byte [prev], 0
mov byte [prev2], 0

.read_loop:

```

```
mov ah, 0x00
int 0x16
cmp al, 0
je .read_loop
```

```
; Выводим символ
mov ah, 0x0E
mov bl, al
xor bh, bh
int 0x10
```

```
; Обновляем буфер
mov al, [prev]
mov [prev2], al
mov al, [last]
mov [prev], al
mov [last], bl
```

```
cmp byte [last], 'm'
jne .check_std
cmp byte [prev], 'b'
jne .check_std
```

```
mov byte [boot_mode], 0
popa
ret
```

.check_std:

```
cmp byte [last], 'd'
jne .read_loop
cmp byte [prev], 't'
jne .read_loop
cmp byte [prev2], 's'
jne .read_loop
```

```
mov byte [boot_mode], 1
popa
ret
```

gdt:

; Нулевой дескриптор

db 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00

; Сегмент кода: base=0, size=4Gb, P=1, DPL=0, S=1(user),

; Type=1(code), Access=00A, G=1, B=32bit

db 0xff, 0xff, 0x00, 0x00, 0x00, 0x9A, 0xCF, 0x00

; Сегмент данных: base=0, size=4Gb, P=1, DPL=0, S=1(user),

; Type=0(data), Access=0W0, G=1, B=32bit

db 0xff, 0xff, 0x00, 0x00, 0x00, 0x92, 0xCF, 0x00

gdt_info: ; Данные о таблице GDT (размер, положение в памяти)

dw gdt_info - gdt - 1 ; Размер таблицы (2 байта)

dw gdt, 0 ; 32-битный физический адрес таблицы.

gdt_end:

turn_protected:

; Отключение прерываний

cli

; Загрузка размера и адреса таблицы дескрипторов

lgdt [gdt_info] ; Для GNU assembler должно быть "lgdt gdt_info"

; Включение адресной линии A20

in al, 0x92

or al, 2

out 0x92, al

; Установка бита PE регистра CR0 - процессор перейдет в защищенный режим

mov eax, cr0

or al, 1

mov cr0, eax

jmp 0x8:protected_mode ; "Дальний" переход для загрузки корректной информации в cs (архитектурные особенности не позволяют этого сделать напрямую).

use32

protected_mode:

; Загрузка селекторов сегментов для стека и данных в регистры

```

    mov ax, 0x10 ; Используется дескриптор с номером 2 в GDT
    mov es, ax
    mov ds, ax
    mov ss, ax
    mov esp, 0x20000

    xor eax, eax
    mov al, [boot_mode]
    mov [0x500], eax

    ; Передача управления загруженному ядру
    jmp 0x08:0x10000 ; Адрес равен адресу загрузки в случае если
ядро скомпилировано в "плоский" код

    jmp $

disk_error:
    mov si, error_msg
    call print_string
    jmp $

print_string:
    mov ah, 0x0E
.next_char:
    lodsb
    test al, al
    jz .done
    int 0x10
    jmp .next_char
.done:
    ret

msg db "Bootloader starting...", 13, 10, 0
error_msg db "Disk error!", 13, 10, 0
load_ok_msg db "Kernel loaded!", 13, 10, 0
last db 0
prev db 0
prev2 db 0
boot_mode db 0

```

```
; Заполнение до 510 байт
times 510 - ($ - $$) db 0
dw 0xAA55
```

Приложение 3

main.cpp

```
#include "kernel.h"

__asm("jmp kmain");

struct idt_entry g_idt[256]; // Реальная таблица IDT
struct idt_ptr g_idtp;      // Описатель таблицы для команды lidt

// Пустой обработчик прерываний.
void default_intr_handler()
{
    asm("pusha");
    // TODO обработчик прерываний
    asm("popa; leave; iret");
}

void intr_reg_handler(int num, unsigned short segm_sel, unsigned
short flags,
                     intr_handler hndlr)
{
    unsigned int hndlr_addr = (unsigned int)hndlr;

    g_idt[num].base_lo = (unsigned short)(hndlr_addr & 0xFFFF);
    g_idt[num].segm_sel = segm_sel;
    g_idt[num].always0 = 0;
    g_idt[num].flags = flags;
    g_idt[num].base_hi = (unsigned short)(hndlr_addr >> 16);
}

void intr_init()
{
    int i;
    int idt_count = sizeof(g_idt) / sizeof(g_idt[0]);
```

```

    for (i = 0; i < idt_count; i++)
    {
        if (i != 0x09) // только клавиатура
            intr_reg_handler(i, GDT_CS, 0x80 | IDT_TYPE_INTR,
                            default_intr_handler);
    }
}

void intr_start()
{
    int idt_count = sizeof(g_idt) / sizeof(g_idt[0]);

    g_idtp.base = (unsigned int)(&g_idt[0]);
    g_idtp.limit = (sizeof(struct idt_entry) * idt_count) - 1;

    asm("lidt %0" : : "m"(g_idtp));
}

void intr_enable() { asm("sti"); }

void intr_disable() { asm("cli"); }

bool bm_mode;

extern "C" int kmain()
{
    const char* hello = "Welcum to GoydaOS!\n";

    clear_screen(0x07);
    print_const_str(hello);

    int boot_choice = BOOT_MODE_ADDR;

    bool bm = (boot_choice == 0) ? 1 : 0;

    bm_mode = bm;

    intr_disable();
    intr_init();
    keyb_init();
}

```

```

    intr_start();
    intr_enable();

    while (1)
    {
        asm("hlt");
    }

    return 0;
}

```

kernel.h

```

#ifndef KERNEL_H
#define KERNEL_H

#define VIDEO_BUF_PTR (0xb8000)
#define IDT_TYPE_INTR (0x0E)
#define PIC1_PORT (0x20)
#define CURSOR_PORT (0x3D4)
#define VIDEO_WIDTH (80)
#define GDT_CS (0x8)
#define BOOT_MODE_ADDR (*(int*)0x500)

typedef enum
{
    KEY_NONE = 0,
    KEY_ESC = 1,
    KEY_1 = 2,
    KEY_2 = 3,
    KEY_3 = 4,
    KEY_4 = 5,
    KEY_5 = 6,
    KEY_6 = 7,
    KEY_7 = 8,
    KEY_8 = 9,
    KEY_9 = 10,
    KEY_0 = 11,
    KEY_MINUS = 12,

```

KEY_EQUAL = 13,
KEY_BACKSPACE = 14,
KEY_TAB = 15,
KEY_Q = 16,
KEY_W = 17,
KEY_E = 18,
KEY_R = 19,
KEY_T = 20,
KEY_Y = 21,
KEY_U = 22,
KEY_I = 23,
KEY_O = 24,
KEY_P = 25,
KEY_LBRACKET = 26,
KEY_RBRACKET = 27,
KEY_ENTER = 28,
KEY_LCTRL = 29,
KEY_A = 30,
KEY_S = 31,
KEY_D = 32,
KEY_F = 33,
KEY_G = 34,
KEY_H = 35,
KEY_J = 36,
KEY_K = 37,
KEY_L = 38,
KEY_SEMICOLON = 39,
KEY_APOSTROPHE = 40,
KEY_GRAVE = 41,
KEY_LSHIFT = 42,
KEY_BACKSLASH = 43,
KEY_Z = 44,
KEY_X = 45,
KEY_C = 46,
KEY_V = 47,
KEY_B = 48,
KEY_N = 49,
KEY_M = 50,
KEY_COMMA = 51,
KEY_DOT = 52,

```
KEY_SLASH = 53,  
KEY_RSHIFT = 54,  
KEY_KP_MULT = 55, // клавиатура numpad  
KEY_LALT = 56,  
KEY_SPACE = 57,  
KEY_CAPSLOCK = 58,  
KEY_F1 = 59,  
KEY_F2 = 60,  
KEY_F3 = 61,  
KEY_F4 = 62,  
KEY_F5 = 63,  
KEY_F6 = 64,  
KEY_F7 = 65,  
KEY_F8 = 66,  
KEY_F9 = 67,  
KEY_F10 = 68,  
KEY_NUMLOCK = 69,  
KEY_SCROLLLOCK = 70,  
KEY_KP7 = 71,  
KEY_KP8 = 72,  
KEY_KP9 = 73,  
KEY_KP_MINUS = 74,  
KEY_KP4 = 75,  
KEY_KP5 = 76,  
KEY_KP6 = 77,  
KEY_KP_PLUS = 78,  
KEY_KP1 = 79,  
KEY_KP2 = 80,  
KEY_KP3 = 81,  
KEY_KP0 = 82,  
KEY_KP_DOT = 83,  
KEY_F11 = 87,  
KEY_F12 = 88,  
KEY_HOME = 71, // пример, можно уточнить точные сканкоды стрелок
```

и навигации

```
KEY_UP = 72,  
KEY_PAGEUP = 73,  
KEY_LEFT = 75,  
KEY_RIGHT = 77,  
KEY_END = 79,
```

```

    KEY_DOWN = 80,
    KEY_PAGEDOWN = 81,
    KEY_INSERT = 82,
    KEY_DELETE = 83
    // остальное можно добавить по необходимости
} Keycode;

extern bool bm_mode;

extern unsigned int curs_x, curs_y;

extern unsigned char* templ;

// Функции ввода-вывода
static inline unsigned char inb(unsigned short port)
{
    unsigned char data;
    asm volatile("inb %w1, %b0" : "=a"(data) : "Nd"(port));
    return data;
}

static inline void outb(unsigned short port, unsigned char data)
{
    asm volatile("outb %b0, %w1" : : "a"(data), "Nd"(port));
}

static inline void outw(unsigned short port, unsigned short data)
{
    asm volatile("outw %w0, %w1" : : "a"(data), "Nd"(port));
}

// Функции работы с экраном
void cursor_moveto(unsigned int strnum, unsigned int pos);
void clear_screen(int color);
void out_str(int color, unsigned char* str, unsigned int strnum,
             unsigned int pos);

void out_char(int color, unsigned char ch, unsigned int strnum,
              unsigned int pos);
void print_str(unsigned char* ptr);

```

```

void print_const_str(const char* str);

// Функции клавиатуры
void keyboard_machine(int scan_code, bool is_pressed);
void keyb_process_keys();
void keyb_handler();
void keyb_init();

// Функции IDT
// Структура описывает данные об обработчике прерывания
struct idt_entry
{
    unsigned short base_lo; // Младшие биты адреса обработчика
    unsigned short segm_sel; // Селектор сегмента кода
    unsigned char always0; // Этот байт всегда 0
    unsigned char
        flags; // Флаги тип. Флаги: P, DPL, Типы - это константы -
IDT_TYPE...
    unsigned short base_hi; // Старшие биты адреса обработчика
} __attribute__((packed)); // Выравнивание запрещено

// Структура, адрес который передается как аргумент команды lidt
struct idt_ptr
{
    unsigned short limit;
    unsigned int base;
} __attribute__((packed));

typedef void (*intr_handler)();
void intr_reg_handler(int num, unsigned short segm_sel, unsigned
short flags,
                    intr_handler hndlr);
void intr_init();
void intr_start();
void intr_enable();
void intr_disable();

// Команды
void command_machine(unsigned char* str);

```

```

// Функции работы со строками
void clear_str(unsigned char* str);
int uc_strcmp(const unsigned char* s1, const char* s2);
void to_upcase(unsigned char* in, unsigned char* out);
void to_downcase(unsigned char* in, unsigned char* out);
int uc_strlen(unsigned char* s);
void uc_strcp(unsigned char* s1, unsigned char* s2);
void int_to_str(int value, unsigned char* buffer);
int boyer_moore_search(unsigned char* text);
void print_bm_table_simple(unsigned char* pattern);
int std_search(unsigned char* text);

#endif

```

screen.cpp

```

#include "kernel.h"

unsigned int curs_x = 0, curs_y = 0;

void cursor_moveto(unsigned int strnum, unsigned int pos)
{
    unsigned short new_pos = strnum * VIDEO_WIDTH + pos;

    curs_x = pos;
    curs_y = strnum;

    outb(CURSOR_PORT, 0x0F);          // индекс младшего байта
    outb(CURSOR_PORT + 1, new_pos & 0xFF); // младший байт

    outb(CURSOR_PORT, 0x0E);          // индекс старшего байта
    outb(CURSOR_PORT + 1, (new_pos >> 8) & 0xFF); // старший байт
}

void clear_screen(int color)
{
    unsigned char* video_buf = (unsigned char*)VIDEO_BUF_PTR;

    for (int i = 0; i < VIDEO_WIDTH * 25; i++)

```

```

{
    video_buf[0] = ' ';
    video_buf[1] = color;

    video_buf += 2;
}

cursor_moveto(0, 0);
}

void out_str(int color, unsigned char* str, unsigned int strnum,
             unsigned int pos)
{
    unsigned char* video_buf = (unsigned char*)VIDEO_BUF_PTR;
    video_buf += (80 * strnum + pos) *
                2; // В зависимости от номера строки мы указываем
смещение
                // в видеобуфере где будет отображаться наша строка

    unsigned short len = 0;
    while (*str)
    {
        len++;
        if (*str == '\n')
        {
            cursor_moveto(strnum + 1, 0);
            len = 0;
        }
        else
        {
            video_buf[0] = (unsigned char)*str; // Символ(код)
            video_buf[1] = color;               // Цвет символа и фона

            video_buf += 2;
        }

        str++;
    }

    cursor_moveto(curs_y, curs_x + len);
}

```

```
}
```

```
void print_str(unsigned char* ptr) { out_str(0x07, ptr, curs_y,  
curs_x); }
```

```
void print_const_str(const char* str)  
{
```

```
    unsigned int i = 0;
```

```
    unsigned char dest[VIDEO_WIDTH];
```

```
    while (str[i] && i < VIDEO_WIDTH - 1) // оставляем место для  
нуля
```

```
    {  
        dest[i] = (unsigned char)str[i];  
        i++;  
    }
```

```
    dest[i] = 0;
```

```
    print_str(dest);  
}
```

```
void out_char(int color, unsigned char ch, unsigned int strnum,  
              unsigned int pos)
```

```
{  
    unsigned char* video_buf = (unsigned char*)VIDEO_BUF_PTR;  
    video_buf += (VIDEO_WIDTH * strnum + pos) * 2;
```

```
    video_buf[0] = (unsigned char)ch;
```

```
    video_buf[1] = color;
```

```
}
```

strings.cpp

```
#include "kernel.h"
```

```
#define ALPHABET_SIZE 256
```

```
unsigned char* templ;
```

```

void clear_str(unsigned char* str)
{
    if (str == 0)
        return;

    unsigned char* ptr = str;
    while (*ptr != 0)
    {
        *ptr = 0;
        ptr++;
    }
}

int uc_strcmp(const unsigned char* s1, const char* s2)
{
    while (*s1 && *s2)
    {
        if (*s1 != *s2)
            return (int)(*s1) - (int)(*s2);
        s1++;
        s2++;
    }

    return (int)(*s1) - (int)(*s2);
}

void to_upcase(unsigned char* in, unsigned char* out)
{
    clear_str(out);
    int i = 0;
    while (in[i] && i < VIDEO_WIDTH - 1) // защита от переполнения
    {
        unsigned char c = in[i];
        out[i] = (c >= 'a' && c <= 'z') ? c - 32 : c;
        i++;
    }
    out[i] = 0;
}

```

```

void to_lowercase(unsigned char* in, unsigned char* out)
{
    clear_str(out);
    int i = 0;
    while (in[i] && i < VIDEO_WIDTH - 1) // защита от переполнения
    {
        unsigned char c = in[i];
        out[i] = (c >= 'A' && c <= 'Z') ? c + 32 : c;
        i++;
    }
    out[i] = 0;
}

```

```

int uc_strlen(unsigned char* s)
{
    int len = 0;
    while (s[len] != 0)
        len++;
    return len;
}

```

```

void uc_strcp(unsigned char* dst, unsigned char* src)
{
    unsigned char* start = dst;

    while (*src != 0)
    {
        *dst = *src;
        dst++;
        src++;
    }

    *dst = 0; // копируем ноль-терминатор
    dst = start;
}

```

```

void int_to_str(int value, unsigned char* buffer)
{
    int i = 0;
    int is_negative = 0;

```

```
// Обработка отрицательных чисел
if (value < 0)
{
    is_negative = 1;
    value = -value;
}

// Специальный случай для 0
if (value == 0)
{
    buffer[i++] = '0';
    buffer[i] = 0;
    return;
}

// Записываем цифры в обратном порядке
while (value > 0)
{
    int digit = value % 10;
    buffer[i++] = '0' + digit;
    value /= 10;
}

if (is_negative)
    buffer[i++] = '-';

buffer[i] = 0;

// Разворачиваем строку
int start = 0;
int end = i - 1;

while (start < end)
{
    unsigned char tmp = buffer[start];
    buffer[start] = buffer[end];
    buffer[end] = tmp;
    start++;
    end--;
}
```

```

    }
}

int std_search(unsigned char* text)
{
    int n = uc_strlen(text);
    int m = uc_strlen(templ);

    if (m == 0)
        return 0;
    if (m > n)
        return -1;

    // Перебираем все возможные начала
    for (int i = 0; i <= n - m; i++)
    {
        int j;
        // Сравниваем шаблон с частью текста, начиная с i
        for (j = 0; j < m; j++)
        {
            if (templ[j] != text[i + j])
            {
                break; // Не совпало
            }
        }
        if (j == m)
        {
            return i; // Нашли! Все символы совпали
        }
        // Если не нашли, цикл переходит к следующему i (сдвиг на 1)
    }
    return -1; // Не нашли
}

```

```

int boyer_moore_search(unsigned char* text)
{
    int n = uc_strlen(text);
    int m = uc_strlen(templ);

    if (m == 0)

```

```

    return 0;
if (m > n)
    return -1;

int badchar[ALPHABET_SIZE] = {-1};

for (int i = 0; i < m; i++)
{
    badchar[(unsigned char)templ[i]] = i;
}

int shift = 0;

while (shift <= n - m)
{
    int j = m - 1;

    while (j >= 0 && templ[j] == text[shift + j])
        j--;

    if (j < 0)
    {
        return shift;
    }
    else
    {
        int bad_char_index = badchar[(unsigned char)text[shift +
j]];

        int shift_by_bad_char = j - bad_char_index;

        if (shift_by_bad_char < 1)
            shift_by_bad_char = 1;

        shift += shift_by_bad_char;
    }
}

return -1;
}

```

```

void print_bm_table_simple(unsigned char* pattern)
{
    int m = uc_strlen(pattern);

    if (m == 0)
    {
        print_const_str("Empty pattern\n");
        return;
    }

    print_const_str("BM info: ");

    int printed[ALPHABET_SIZE] = {0};

    for (int i = 0; i < m; i++)
    {
        unsigned char c = pattern[i];

        if (!printed[c])
        {
            printed[c] = 1;

            int shift;

            int last_occ = i;
            for (int j = i + 1; j < m; j++)
            {
                if (pattern[j] == c)
                {
                    last_occ = j;
                }
            }

            if (last_occ == m - 1)
            {
                shift = m;
            }
            else
            {

```

```

    shift = m - 1 - last_occ;
    if (shift == 0)
        shift = 1; // Минимальный сдвиг = 1
}

unsigned char char_str[2] = {c, 0};
print_str(char_str);
print_const_str(":");

unsigned char shift_str[VIDEO_WIDTH];
int_to_str(shift, shift_str);
print_str(shift_str);

print_const_str(" ");
}
}

print_const_str("\n");
}

```

keyboard.cpp

```
#include "kernel.h"
```

```

unsigned char
scan_table[128] =
{
    0,    27,  '1', '2', '3', '4', '5', '6', '7', '8',
    '9',  '0',  '-', '=', '\b', '\t', 'q', 'w', 'e', 'r',
    't',  'y', 'u', 'i', 'o', 'p',  '[', ']', '\n', 0, //
LCtrl=29
    'a',  's', 'd', 'f', 'g',  'h',  'j', 'k', 'l',  ';',
    '\'', '`', 0, // LShift=42
    '\\', 'z', 'x', 'c', 'v',  'b',  'n', 'm', ',', '.',
    '/',  0,  0, 0,  ' ',  0,  0,
    ' ' // пробел на 57
    // Остальные элементы автоматически 0
};

```

```

void keyboard_machine(int scan_code, bool is_pressed)
{
    static bool shift_pressed = false;
    static unsigned char command[40] = {0};
    static unsigned char com_len = 0;

    switch (scan_code)
    {
    case KEY_BACKSPACE:
        if (is_pressed)
        {
            if (curs_x > 0)
            {
                com_len--;
                command[com_len] = 0;
                cursor_moveto(curs_y, curs_x - 1);
                out_char(0x07, '\\0', curs_y, curs_x);
            }
        }
        break;
    case KEY_LSHIFT:
        shift_pressed = is_pressed;
        break;
    case KEY_ENTER:
        if (is_pressed)
        {
            if (curs_y < 24)
            {
                cursor_moveto(curs_y + 1, 0);
            }
            else
            {
                clear_screen(0x07);
            }

            command_machine(command);
            clear_str(command);
            com_len = 0;
        }
        break;
    }
}

```

```
default:
    if (is_pressed && com_len < 40)
    {
        unsigned char c = 0;

        if (scan_code < sizeof(scan_table))
            c = scan_table[scan_code];

        if (c != 0 && shift_pressed)
        {
            if (c >= 'a' && c <= 'z')
                c -= 32; // буквы в верхний регистр
            else
            {
                switch (c)
                {
                    case '1':
                        c = '!';
                        break;
                    case '2':
                        c = '@';
                        break;
                    case '3':
                        c = '#';
                        break;
                    case '4':
                        c = '$';
                        break;
                    case '5':
                        c = '%';
                        break;
                    case '6':
                        c = '^';
                        break;
                    case '7':
                        c = '&';
                        break;
                    case '8':
                        c = '*';
                        break;
                }
            }
        }
    }
}
```

```
case '9':
    c = '(';
    break;
case '0':
    c = ')';
    break;
case '-':
    c = '_';
    break;
case '=':
    c = '+';
    break;
case '[':
    c = '{';
    break;
case ']':
    c = '}';
    break;
case '\\':
    c = '|';
    break;
case ';':
    c = ':';
    break;
case '\\':
    c = '"';
    break;
case ',':
    c = '<';
    break;
case '.':
    c = '>';
    break;
case '/':
    c = '?';
    break;
}
}
}
```

```

    if (c == 0)
        c = '?'; // для наглядности, если символ пуст

    command[com_len] = c;
    com_len++;

    out_char(0x07, c, curs_y, curs_x);
    cursor_moveto(curs_y, curs_x + 1);
}
break;
}
}

void keyb_process_keys()
{
    if (inb(0x64) & 0x01)
    {
        unsigned char scan_code;
        unsigned char state;

        scan_code = inb(0x60);

        if (scan_code & 0x80)
        {
            scan_code -= 0x80;
            keyboard_machine(scan_code, false);
        }
        else
        {
            keyboard_machine(scan_code, true);
        }
    }
}

void keyb_handler()
{
    asm("pusha");

    keyb_process_keys();
}

```

```
    outb(PIC1_PORT, 0x20);
    asm("popa; leave; iret");
}

void keyb_init()
{
    intr_reg_handler(0x09, GDT_CS, 0x80 | IDT_TYPE_INTR,
keyb_handler);

    outb(PIC1_PORT + 1, 0xFF ^ 0x02);
}
```